Spark Configuration and Performance Evaluation

1. Project Overview and Dataset Description

1.1 Project Overview

The purpose of this project was to evaluate the performance and scalability of Apache Spark when processing the CDC Diabetes Health Indicators dataset. The experiments compared a local Spark configuration with Spark Standalone configurations using 1, 2, and 4 worker processes, with each worker assigned one CPU core.Three major workloads were evaluated: data preprocessing, statistical analysis, and machine learning. Each workload was executed three times under each configuration after an untimed warm-up run. Execution times were recorded and compared using mean runtime, standard deviation, and speedup to examine how the number of Spark executors affected performance.

1.2 Dataset Description

The project used the **CDC Diabetes Health Indicators dataset**, obtained from the UCI Machine Learning Repository.

Dataset source: https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators

The dataset contains 253,680 records and 22 columns, consisting of 21 predictor variables and one target variable, Diabetes_binary. The target identifies whether an individual has diabetes (1) or does not have diabetes (0).The dataset was downloaded using the ucimlrepo Python package and saved as a CSV file for processing by the separate Spark applications.The same dataset and analytical workloads were used across the benchmark configurations to support a consistent comparison of execution performance.

2. Spark Environment and Cluster Configuration

2.1 Computing Environment

The experiments were conducted in Google Colab using its hosted runtime. The environment was inspected before the Spark applications were started to document the available computing resources and software versions.

Table 2.1: Hardware and Software Environment

Computing Environment	Google Colab
Logical CPU count	2

Total host RAM	12.67
Python version	3.13.16
Java version	OpenJDK 21.0.12.1
Apache Spark/PySpark	4.0.4
Scala Version	2.13.16

```
Total host RAM: 12.67 GiB
PySpark / Spark package version: 4.0.4
Python: 3.13.16
Java version:
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-24.04.4-Ubuntu, mixed mode, sharing)
spark-submit version:
WARNING: Using incubator modules: jdk.incubator.vector
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.0.4
      /_/

Using Scala version 2.13.16, OpenJDK 64-Bit Server VM, 21.0.12.1
Branch HEAD
Compiled by user holden on 2026-07-12T04:41:54Z
Revision c7d67e3f5d4c9d88a480367b44fc54d26adf99ab
Url https://github.com/apache/spark
Type --help for more information.

Saved environment information to /content/diabetes_benchmark_environment.json
```

2.2 Spark Installation and Master Configuration

PySpark and the required Python packages were installed in the Colab environment. The Spark Standalone cluster was then initialized by launching a master process using Spark's spark-class command.The master was configured to listen on the local host using the address spark://127.0.0.1:17077. Its web interface was made available on port 18090 to support monitoring of the cluster.The master process remained active while the worker configurations were changed between benchmark experiments.

```
Master PID: 20395 | Master URL: spark://127.0.0.1:17077
Master UI: http://127.0.0.1:18090/json/
```

2.3 Worker and Executor Configuration

The benchmark used separate Java processes to launch Spark workers. Each worker was assigned one CPU core and 1 GB of memory. The worker manager stopped the existing worker

processes before starting a new configuration, ensuring that the intended number of workers was active for each experiment.

Three Spark Standalone configurations were tested:

- **One worker:** 1 worker with 1 core and 1 GB memory.
- **Two workers:** 2 workers, each with 1 core and 1 GB memory.
- **Four workers:** 4 workers, each with 1 core and 1 GB memory.

Each worker was launched as a separate Java process, and one worker was used per executor. This arrangement made it possible to compare the effect of increasing the number of executors while keeping the resources assigned to each worker consistent. The workers were separate processes running on the same Google Colab virtual machine, rather than separate physical machines. Therefore, the four-worker configuration did not provide four independent CPU cores physically available to the experiment. All workers shared the host's two logical CPUs and available memory.

2.4 Baseline and Benchmark Configurations

A local[1] Spark application was used as the baseline. This configuration runs Spark locally using one processing thread. It was compared with the Standalone configurations containing one, two, and four workers.Each benchmark configuration used a fresh Spark application. Before execution, the benchmark job checked the registered executor count to confirm that the requested configuration was active.

Table 2.4: Configurations Used in the Performance Evaluation

Configuration	Execution mode	Processing Resources
Baseline	Local(1)	1 local processing thread
1 Executor	Spark Standalone	1worker, 1 core
2 Executors	Spark Standalone	2 workers, 1 core each
4 Executors	Spark Standalone	4 workers. 1 core each

3. Benchmark Workloads and Experimental Procedure

3.1 Benchmark Workloads

Three workloads were selected to evaluate different types of Spark processing. The operations were based on the analysis developed for the diabetes project and were executed under each Spark configuration.

Table 3.1: Benchmark Workloads

Workload	Operations	Purpose
Data Preprocessing	CSV loading,schema inference,column validation, missing values, remove duplicates, data type conversion and record counting	Measure the time required to prepare the dataset for analysis
Statistical Analysis	Diabetes prevalence, descriptive statistics, BMI and age summaries, risk-factor frequency counts, and Pearson correlation	Measure Spark's performance when performing analytical and aggregation operations
Machine Learning	Feature assembly, standardization and logistic regression model fitting	Measure the time required to train the prediction model

The preprocessing workload included reading the CSV file into Spark, checking the required columns, counting records, checking for missing values, removing duplicates, and converting columns to numeric types.The statistical analysis workload included grouped calculations and descriptive summaries, followed by Pearson correlation analysis across the selected variables.The machine learning workload used a Spark MLlib pipeline consisting of VectorAssembler, StandardScaler, and LogisticRegression. The logistic regression model was configured with a maximum of 50 iterations and a regularization parameter of 0.01.

3.2 Experimental Procedure

The same dataset and benchmark operations were used across the four execution configurations: local[1], one executor, two executors, and four executors.For each configuration, the benchmark application first verified the expected executor count. The training and testing datasets were prepared using an 80:20 random split with a seed of 42. Both datasets were cached before the timed machine learning operations.Each workload was executed once as a warm-up operation. The warm-up execution was not included in the performance measurements. After the warm-up, each workload was executed three additional times, and its execution time was recorded.This procedure was repeated for the local baseline and each Standalone worker configuration. The Spark master remained active while the worker processes were stopped and restarted between configurations.

3.3 Execution Time Measurement

Execution time was measured using Python's time.perf_counter() function. A start time was recorded immediately before each workload, and the elapsed time was calculated when the operation completed.The three measured execution times for each workload and configuration were used to calculate the mean runtime and standard deviation.

The mean runtime was calculated as:

$$[\backslash bar\{T\} = \backslash frac\{1\}\{n\}\backslash sum_\{i = 1\}\^{}\{n\}T_i]$$

where (T_i) represents an individual execution time and (n=3) represents the number of measured runs.Standard deviation was used to describe the variation in execution times across the three repetitions. Lower standard deviation indicates more consistent execution times.

3.4 Performance Metrics

The benchmark used mean execution time, standard deviation, speedup, and efficiency to compare the configurations.Speedup was calculated relative to the one-executor Standalone configuration for each workload:

$$[Speedup = \backslash frac\{T_1\}\{T_n\}]$$

where (T_1) is the mean runtime with one Standalone executor and (T_n) is the mean runtime with the configuration being evaluated.Parallel efficiency was calculated as:

$$[Efficiency = \backslash frac\{Speedup\}\{n\}]$$

where (n) is the number of executors.

We calculated speedup relative to the local[1] baseline. This provided a second comparison between local execution and the Standalone configurations.Model evaluation metrics, including AUC, accuracy, weighted precision, weighted recall, and F1-score, were calculated after the timed training runs. These metrics were used to check whether the model produced consistent predictive results across the different configurations; they were not included in the measured model-training time.

Baseline timings

```
Warm-up: Preprocessing
Warm-up: Statistical Analysis
Warm-up: Machine Learning
Warm-up complete
Preprocessing | 1 executors | Run 1: 3.618s
Preprocessing | 1 executors | Run 2: 3.253s
Preprocessing | 1 executors | Run 3: 3.330s
Statistical Analysis | 1 executors | Run 1: 17.789s
Statistical Analysis | 1 executors | Run 2: 16.243s
Statistical Analysis | 1 executors | Run 3: 15.273s
Machine Learning | 1 executors | Run 1: 4.811s
Machine Learning | 1 executors | Run 2: 4.798s
Machine Learning | 1 executors | Run 3: 5.125s
Test metrics: {'AUC': 0.8060847892440575, 'accuracy': 0.8487215289292259, 'weightedPrecision': 0.813933156793963, 'weightedRecall': 0.8487215289292259, 'f1': 0.8064202236826045}
```

	Method	Executors	Cores_Per_Executor	Actual_Executors	Mode	Repeat	Time_Seconds
0	Preprocessing	1	1	1	local[1]	1	3.6176
1	Preprocessing	1	1	1	local[1]	2	3.2528
2	Preprocessing	1	1	1	local[1]	3	3.3303
3	Statistical Analysis	1	1	1	local[1]	1	17.7887
4	Statistical Analysis	1	1	1	local[1]	2	16.2426
5	Statistical Analysis	1	1	1	local[1]	3	15.2727
6	Machine Learning	1	1	1	local[1]	1	4.8112
7	Machine Learning	1	1	1	local[1]	2	4.7985
8	Machine Learning	1	1	1	local[1]	3	5.1248

4. Benchmark Results and Performance Comparison

4.1 Execution Time Results

The execution times were collected over three measured runs for each workload and configuration. The mean and standard deviation were calculated to compare average performance and consistency.

Table 4.1: Mean Execution Time and Standard Deviation

	Method	Executors	Mean_Time	SD_Time	Speedup	Efficiency
0	Machine Learning	1	4.868	0.393	1.000	1.000
1	Machine Learning	2	5.653	0.565	0.861	0.431
2	Machine Learning	4	7.558	0.843	0.644	0.161
3	Preprocessing	1	4.007	0.281	1.000	1.000
4	Preprocessing	2	4.276	0.524	0.937	0.468
5	Preprocessing	4	5.259	0.776	0.762	0.190
6	Statistical Analysis	1	19.597	2.299	1.000	1.000
7	Statistical Analysis	2	19.848	3.072	0.987	0.494
8	Statistical Analysis	4	26.651	1.902	0.735	0.184

The results show that increasing the number of workers did not reduce execution time. The local[1] configuration had the shortest average runtime for preprocessing and statistical

analysis. For machine learning, the one-executor Standalone configuration was slightly faster than the local baseline, although the difference was small.The four-executor configuration recorded the longest average execution time across all three workloads.

4.2 Speedup and Parallel Efficiency

Speedup was calculated relative to the one-executor Standalone configuration. A speedup above 1 would indicate improved performance, while a value below 1 indicates that the configuration was slower than the one-executor configuration.

Table 4.2: Speedup and Parallel Efficiency

Workload	Executor	Speedup & Parallel Efficiency %
Preprocessing	1	1.000(100)
Preprocessing	2	0.937(46.8)
Preprocessing	4	0.762(19)
Statistical analysis	1	1.000(100)
Statistical analysis	2	0.987(49.4)
Statistical analysis	4	0.735(18.4)
Machine Learning	1	1.000(100)
Machine Learning	2	0.861(43.1)
Machine Learning	4	0.644(16.1)

The two-executor configuration performed slightly slower than the one-executor configuration for all three workloads. The difference was particularly small for statistical analysis, where the speedup was 0.987. Performance declined further with four executors.Parallel efficiency also decreased as the number of executors increased. This indicates that adding workers did not produce proportional performance improvements in the available computing environment.

4.3 Comparison with the Local Baseline

The local[1] configuration was used as an additional reference point to determine whether Spark Standalone execution provided a performance advantage over local execution.

Table 4.3: Standalone Runtime Compared with Local[1]

Standalone results compared with local[1] baseline:

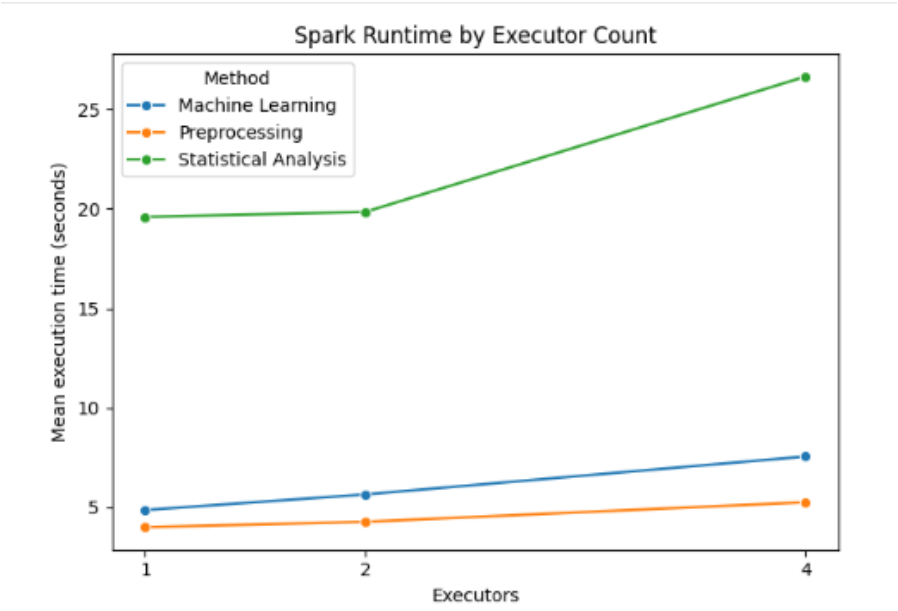
	Method	Executors	Mean_Time	SD_Time	Speedup	Efficiency	Local1_Mean_Time	Speedup_vs_Local1
0	Machine Learning	1	4.868	0.393	1.000	1.000	4.912	1.009
1	Machine Learning	2	5.653	0.565	0.861	0.431	4.912	0.869
2	Machine Learning	4	7.558	0.843	0.644	0.161	4.912	0.650
3	Preprocessing	1	4.007	0.281	1.000	1.000	3.400	0.849
4	Preprocessing	2	4.276	0.524	0.937	0.468	3.400	0.795
5	Preprocessing	4	5.259	0.776	0.762	0.190	3.400	0.647
6	Statistical Analysis	1	19.597	2.299	1.000	1.000	16.435	0.839
7	Statistical Analysis	2	19.848	3.072	0.987	0.494	16.435	0.828
8	Statistical Analysis	4	26.651	1.902	0.735	0.184	16.435	0.617

For preprocessing and statistical analysis, local execution was faster than all Standalone configurations. For machine learning, one Standalone executor achieved a marginal improvement over the local baseline, with a mean runtime of 4.868 seconds compared with 4.911 seconds.Overall, the results indicate that Standalone execution did not provide a consistent performance advantage over local execution for this dataset and hardware environment.

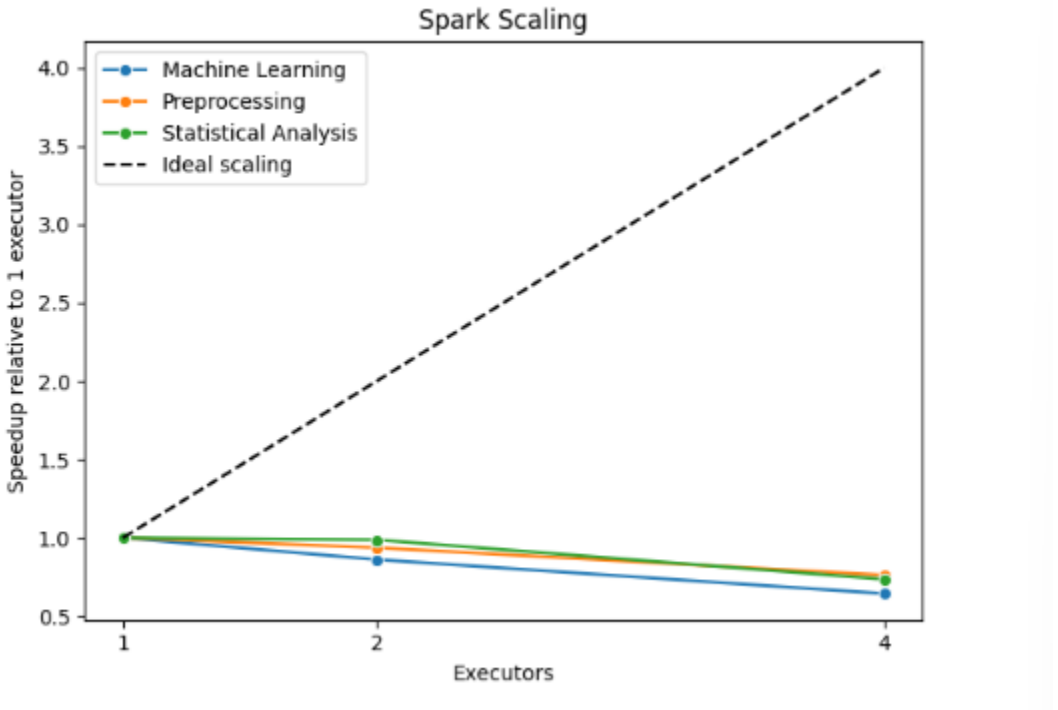
4.4 Runtime and Speedup Visualizations

Two plots were generated from the benchmark summary:

- Figure 4.1: Spark Runtime by Executor Count – compares mean execution time across the one-, two-, and four-executor configurations for each workload.



- Figure 4.2: Spark Scaling – compares measured speedup with ideal linear scaling.



4.5 Interpretation of Results

The benchmark results show that increasing the number of Spark workers did not improve execution performance in this experiment. One important factor was the limited computing environment. Although the four-worker configuration launched four separate worker processes, all workers shared the same Google Colab runtime, which provided only two logical CPUs.The additional workers therefore introduced process scheduling, communication, and coordination overhead without providing additional physical CPU capacity. These overheads may have contributed to the longer execution times observed with two and four workers.

Limitations

- Limited CPU resources: Your Colab environment reported only two logical CPUs. Running four one-core Spark executors on this machine does not provide four independent physical cores. Instead, the executor processes must compete for limited CPU resources.
- Spark coordination overhead: Multiple executors introduce additional communication, scheduling, and coordination costs and this could have slowed the processes given that we used the CPU instead instead of the VMs with GPUs
- Data shuffling and repeated operations: Grouping, aggregation and correlation analysis can involve data movement and repeated Spark jobs which may increase overhead.
- Memory and resource contention: Multiple JVM processes share the same Colab host memory and CPU resources.

6. Conclusion

The performance evaluation demonstrated that Apache Spark can be configured to run data preprocessing, statistical analysis, and machine learning workloads using both local execution and Spark Standalone workers.However, increasing the number of workers from one to four did not improve execution time in the Google Colab environment. The local[1] configuration performed best for preprocessing and statistical analysis, while the one-executor Standalone configuration was marginally faster for machine learning. The additional worker processes introduced overhead while sharing the limited computing resources available in the runtime.The model evaluation results remained consistent across the Standalone configurations, indicating that changes in executor count affected execution time but did not change the reported predictive performance.Overall, the experiment demonstrated that Spark performance depends not only on the number of workers but also on available hardware, workload characteristics, and execution overhead. Future evaluations using larger datasets and dedicated multi-core or distributed environments could provide a more meaningful assessment of Spark's scalability.